

Predictive Storage Caching and Tiered Data Migration

GROUP 10

Nilay Sadanand Borkar: 2023A7PS0401G

Ishan Kaustubh Mehta: 2023A7PS0379G

Outline

1. Overview
2. Work Done Till Milestone 2
3. Milestone 3 (Plan & Results)
4. Next Steps
5. Questions

Overview

Overview

There is an upper bound on the amount of the Cache memory available to us, and this memory is not enough to hold all the KV Cache generated during the prefill phase.

Thus, we use tiered caching. But there is catch! Moving KV pairs between different tiers of storage comes with a **time cost**. Also since cache fetching operations are **reactive**, meaning the cache is fetched from storage after the Model requests it, we halt LLM's forward pass while data is fetched from disk to CPU.

- We attempted to run some experiments how LMCache can help in migration of cache between tiered storages
- Then we tried to build an RL model that reactively learns to migrate the predicted chunks of cache in the *warmer* tiers.

Work Done till Milestone 2

Simulations & Experiments

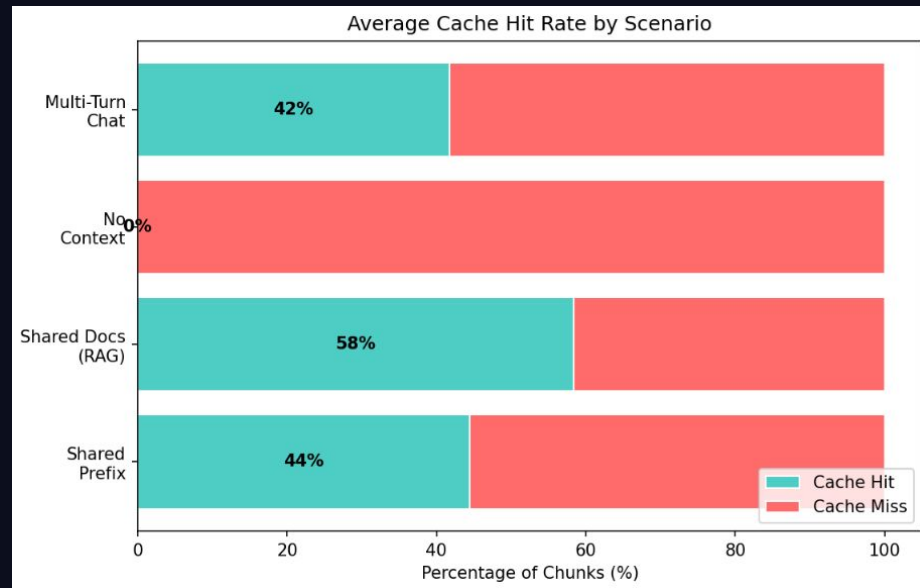
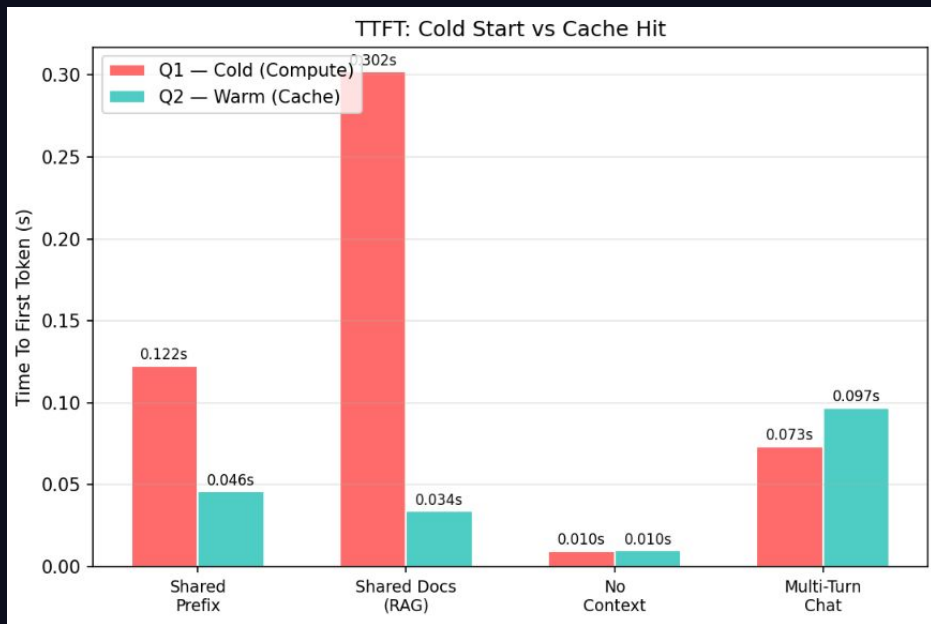
Utilised LMCache, a tiered storage caching system which uses, GPU, Memory and Storage to store key value pairs.

Simulation: First we used some heuristic values to simulate how such a cache would perform during this.

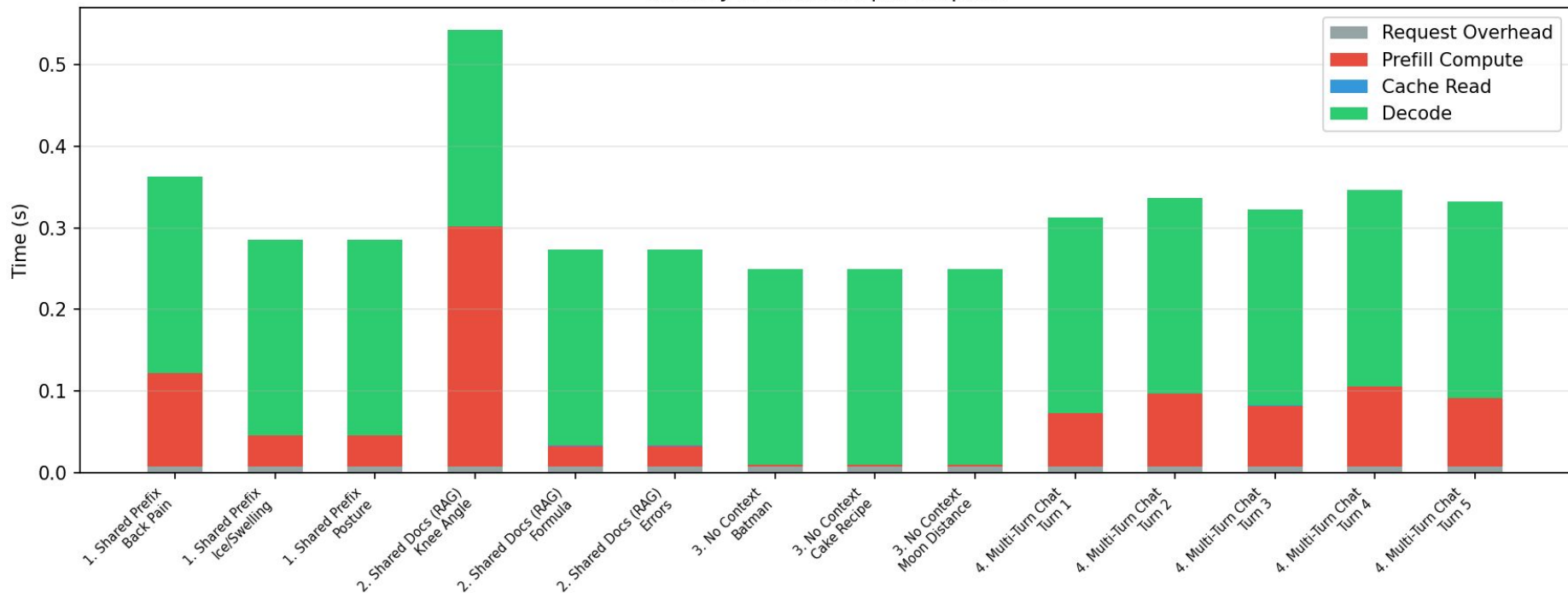
Experiment: Tested the LM Cache model in reality on NVIDIA GPU (using vLLM library to test it)

Tested QWEN 2.5 0.5B parameter model on 4 different different workloads (Prefix, RAG, No context, Multiturn chat) with 50 prompts each

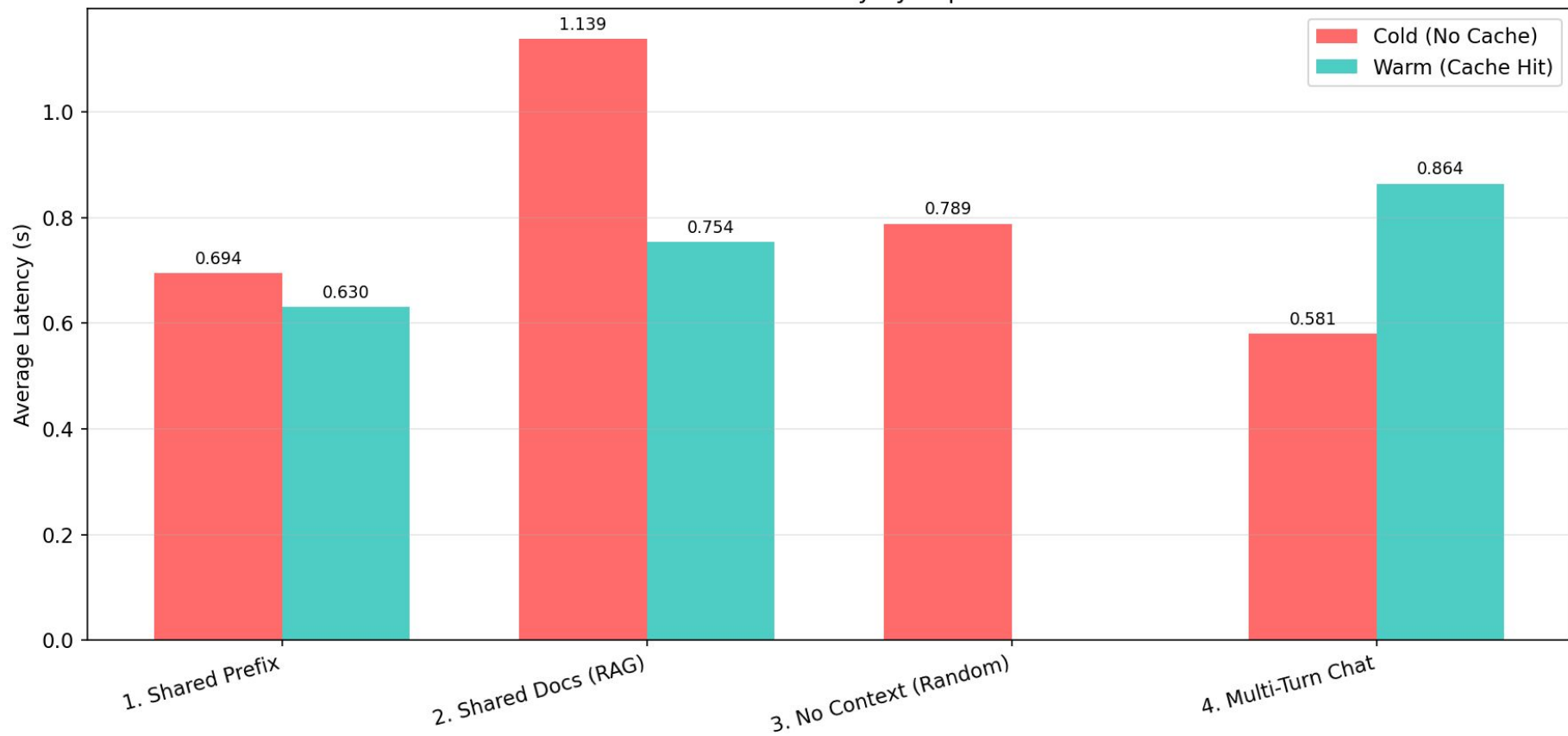
Some Previous Results:



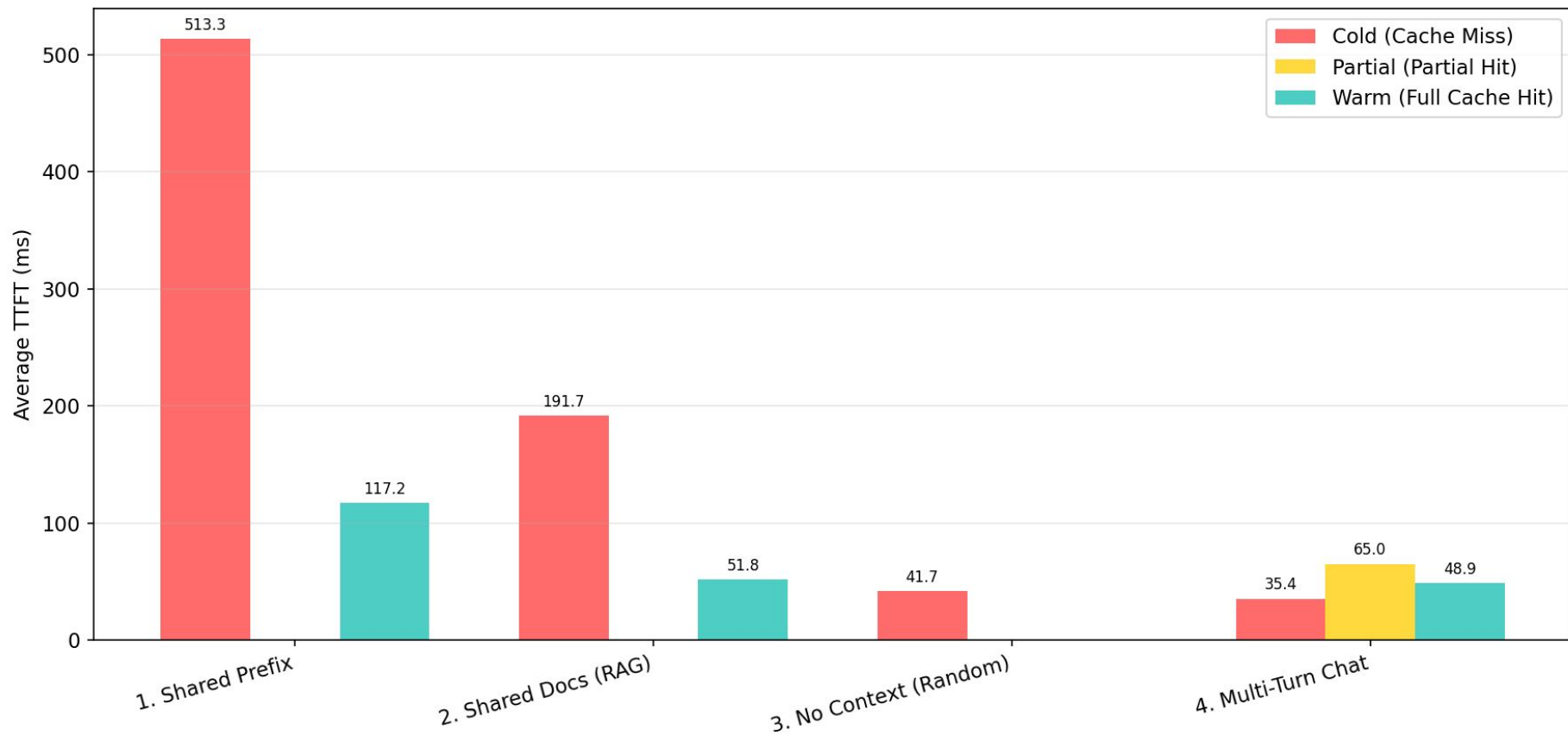
Latency Breakdown per Request

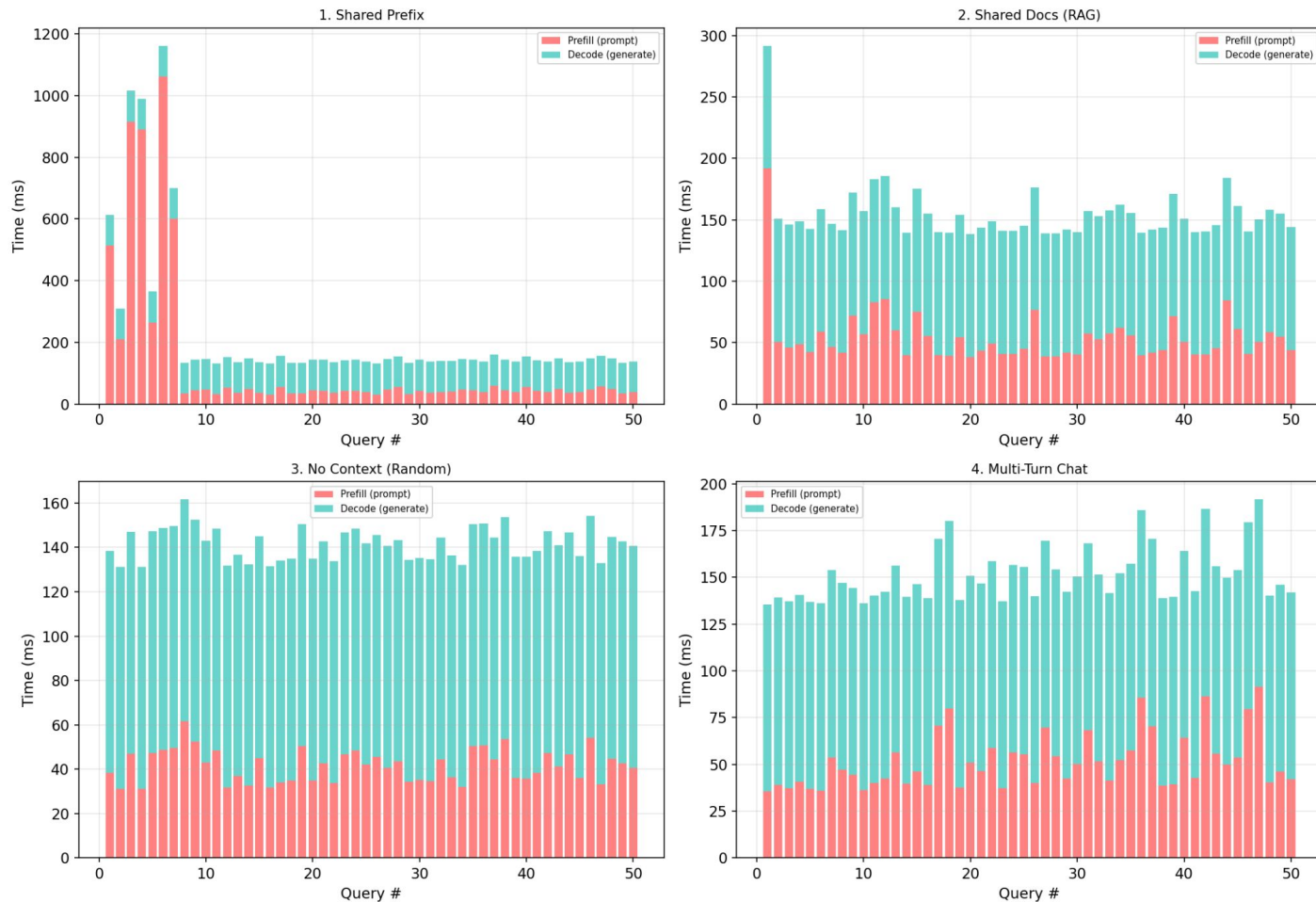


Cold vs Warm Latency by Experiment

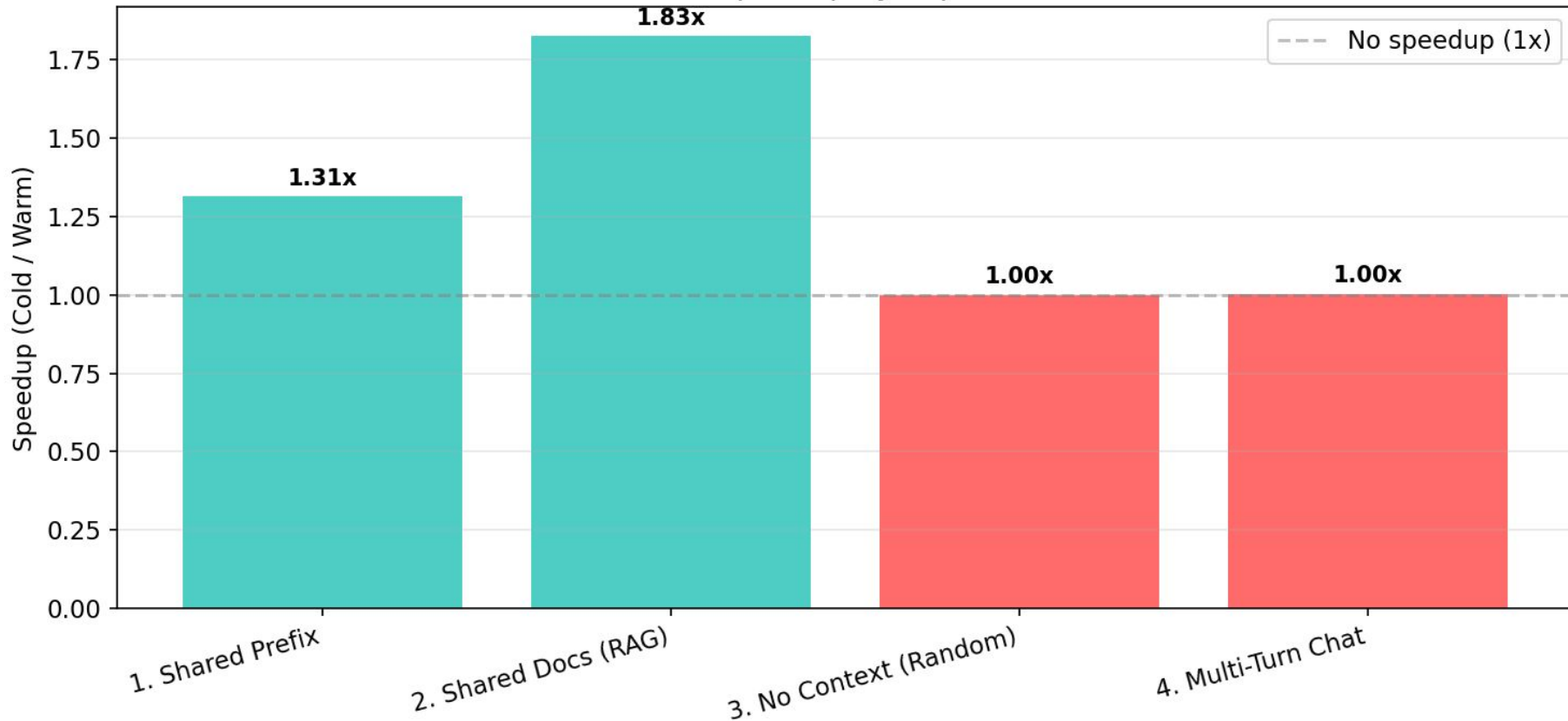


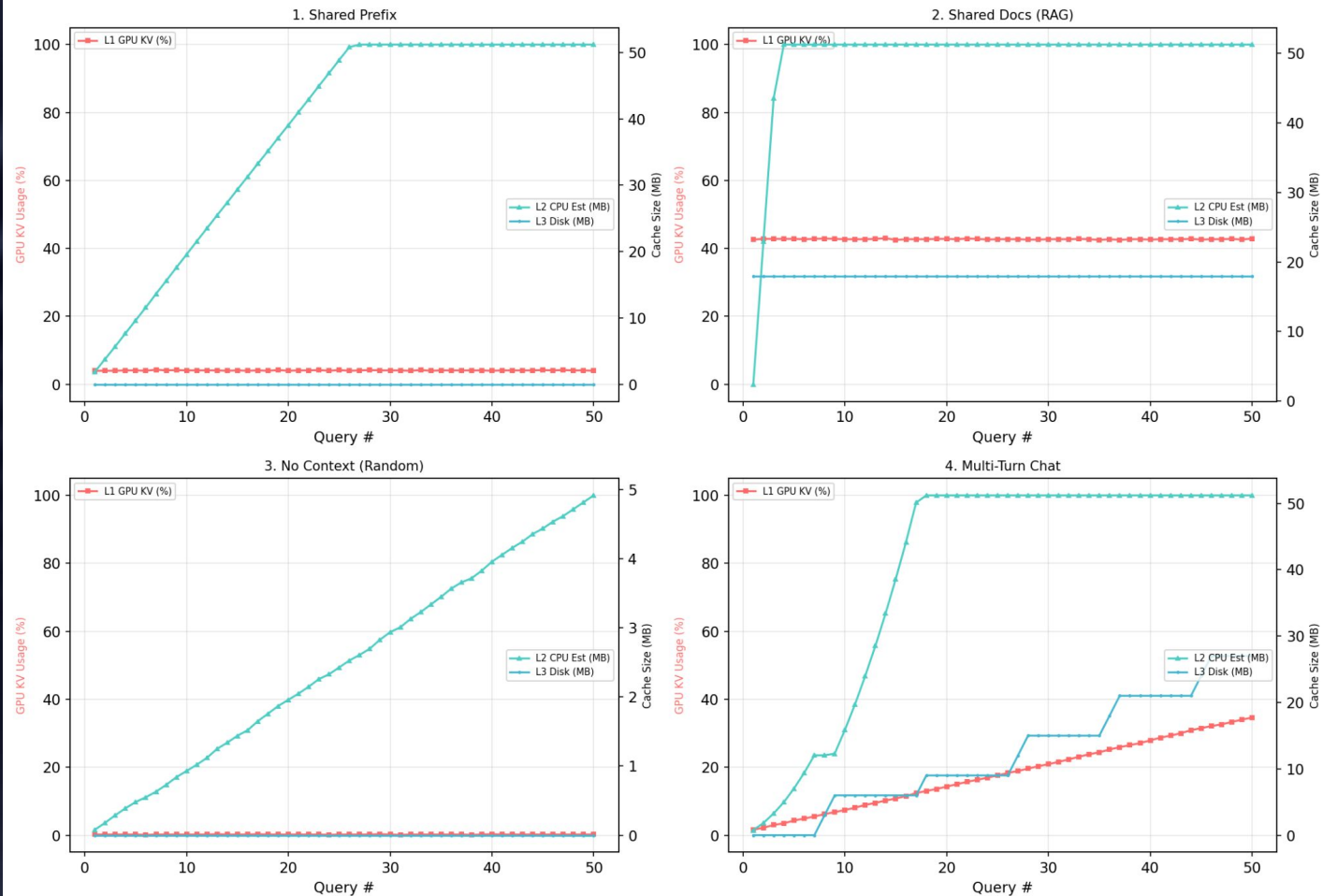
Time To First Token — Cold / Partial / Warm

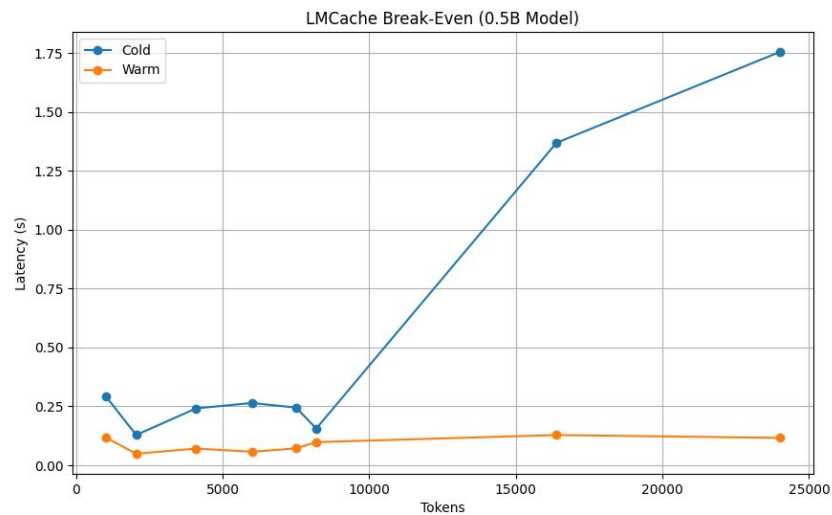
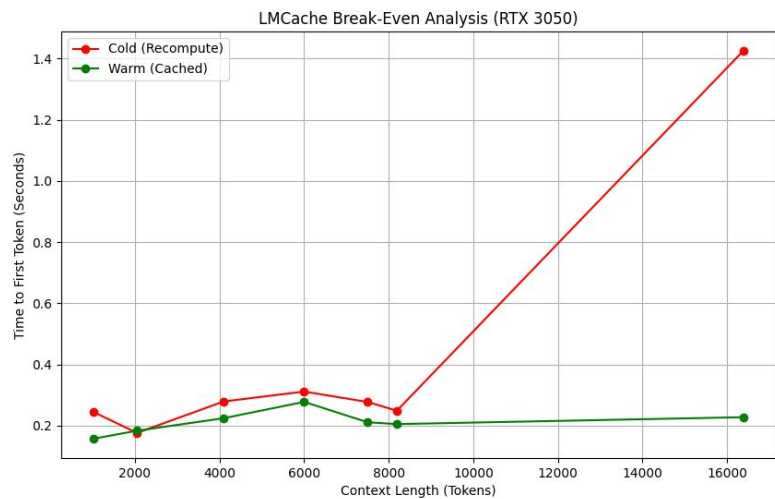




Cache Speedup by Experiment







What we learnt

So, the logical step is to know what part of the cache will be needed, so that you can preload the KV pairs from the cache . Now the problem with this is that we don't know what the next prompt will be.

User prompts also don't necessarily follow the principles of temporal or spatial locality since the next query or question can be from any part of the existing conversation or a totally new question.

How do we deal with this?

Work Done in Milestone 3

Earlier Plan: Neural Cache Prefetcher

Instead of waiting for a cache miss, we aim to predict which `KV-cache` chunks will be needed based on “history of prompts”.

- Generate an “access sequence” trace by logging which `CACHE_CHUNKS` were requested during a session.
For e.g. (`Session_ID`, `Timestamp`, `Current_Query_Embedding`, `Requested_Chunk_ID`)
- Use a `Sentence-Transformer` (e.g. `all-MiniLM-L6-v2`) to convert the input prompt into a vector for the embedding layer.
- Use an `RNN` (or `LSTM`) which takes a sequence of last 3-5 query embeddings and output a probability distribution over available KV-cache chunks in `L3`.
- Based on the prediction, we aim to migrate the required KV-cache data into `L2` before the LLM finishes the prefill phase.

Earlier Plan: Problem!

- LSTM is “**predictive**” - requires **complex periodic training** to remain accurate, and **computationally expensive** requiring high memory bandwidth too.
- Access patterns can shift suddenly, but the model could remain **stale**.
- LSTMs can **fail to capture long-term dependencies** over thousands of accesses - common problem in large scale workloads.

New Plan: Reinforcement Learning!

- RL is “**adaptive**” - a real-time decision making framework, learns through trial & error giving real-time feedback.
- It can be designed with our needs - **reward function** - penalising wasted migration & rewarding TTFT improvements.

RL-Driven Adaptive KV-Cache Tier Migration

The Engine: We leverage a lightweight **Multi-Layer Perceptron (MLP)** trained via **PPO (Proximal Policy Optimisation)**, executing in microseconds so as not to add decision-making latency.

We developed `HardwareCacheEnv`—a custom Gymnasium environment that controls actual data movement on physical hardware (RTX 3050):

- Tier L1 (GPU VRAM): The fastest layer. Where the LLM accesses tensors instantly.
- Tier L2 (Pinned CPU RAM): Intermediate layer. Fast PCIe transfer to GPU.
- Tier L3 (NVMe Disk): The slowest layer, offering massive capacity. The RL Agent strictly manages the proactive migration of chunks from L3 to L2 to prepare them for L1 access.

The Agent (Observation Space)

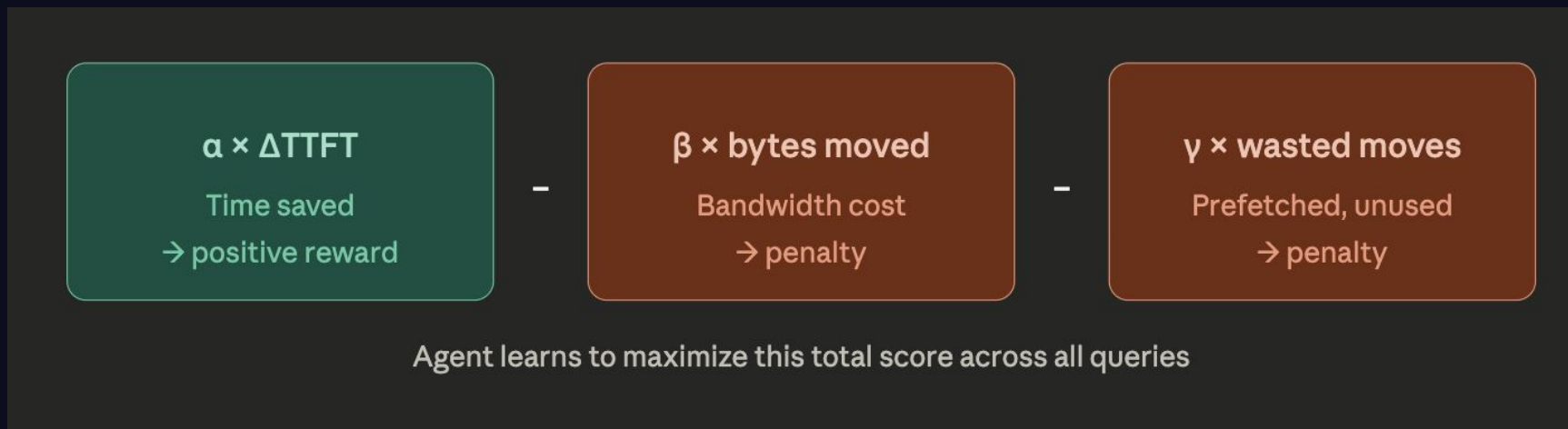
How well RL knows what to fetch? It looks at the **403-dim vector**.

- **Semantic Awareness (384-d)**: We encode the incoming query using a CPU-based `SentenceTransformer`(MiniLM-L6). The agent "reads" the query to understand the context.
- **System Awareness (3-d)**: Real-time normalized utilization metrics of the GPU, CPU, and Disk tiers.
- **Temporal Awareness (16-d)**: Recency scores tracking the top 16 candidate chunks sitting in the L3 Disk cache.

Action Space (Decision to Migrate)

- Agent outputs 16-dim binary vector (`Multibinary(16)`) -> for each of top 16 candidates in L3.
- Selected chunks synchronously trigger disk-to-CPU read operation -> now in L2!

Reward Function: Penalty for Being Wrong



- Positively reward if we prefetched correctly, reducing latency
- ‘Structural’ penalty to the number of MBs moved to warmer tier.
- Harsh penalty to the wastage of bandwidth, because agent moved useless data

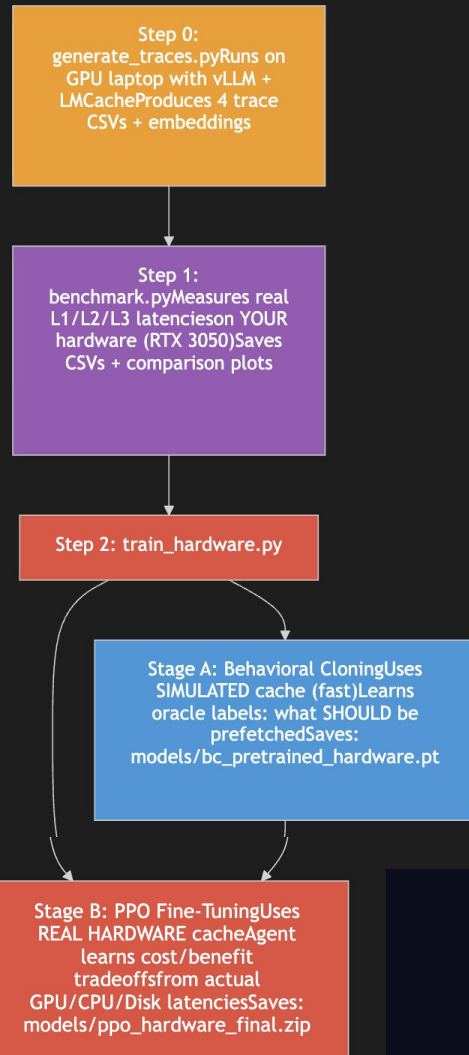
Two-Stage Training Pipeline

1. Behavioral Cloning (BCE Loss)

- Agent mimics an “**Oracle**”, i.e. it knows the future queries & prefetches accordingly
- This teaches the RL to generate the `Multibinary` vectors locally using BCE loss, proves fast & steady.

2. PPO Fine-tuning on Hardware

- Pre-trained Brain now runs on physical `HardwareCacheEnv`
- Trains on true latencies, disk cache migrations, optimising behaviour against real-time.



Running the Experiment

Generating traces

Traces are the logs of which exact KV pairs each query requires. They map these KV pairs to chunks and keep track of them.

Why We Need Traces

- **Enables Offline RL Training:** Trains agents in simulated environments to save on expensive, live GPU resources.
- **Realistic Constraints:** Captures physical hardware limits and performance bounds to mirror real-world production.
- **Accelerates Iteration:** Replays traces to learn optimal prefetching policies thousands of times faster than real-time execution.

Generating traces

How are traces generated?

- **Real-World Execution:** Processes sample prompts through a vLLM engine with LMCache to trigger actual GPU, CPU, and Disk memory movements.
- **Latency & Memory Logging:** Measures the exact Time-To-First-Token (TTFT) and memory footprint to establish "ground truth" hardware performance.
- **Data Chunking:** Algorithmically divides token sequences into fixed-size cache chunks, separating shared data (RAG docs) from unique user prompts.
- **CSV Serialization:** Saves the timing and chunk metadata into a deterministic trace file, allowing simulators to replay the session without active LLM inference.

```
query_id,query_text,input_tokens,chunk_ids_needed,shared_chunk_ids,unique_chunk_ids,num_chunks
1,Who is the founder of Microsoft?,7,[0],[],[0],1
2,What is the chemical symbol for sodium?,8,[1],[],[1],1
3,How many players are there in a baseball team?,10,[2],[],[2],1
4,What year did World War II end?,8,[3],[],[3],1
5,What is the tallest species of tree?,8,[4],[],[4],1
6,Who wrote The Divine Comedy?,6,[5],[],[5],1
7,What is the hardest rock type?,7,[6],[],[6],1
8,How many hearts does a squid have?,8,[7],[],[7],1
9,What is the main ingredient in guacamole?,10,[8],[],[8],1
10,Who developed the polio vaccine?,7,[9],[],[9],1
11,What is the longest-running Broadway show?,8,[10],[],[10],1
12,What is the square of 25?,9,[11],[],[11],1
13,Who was the first woman to win a Nobel Prize?,11,[12],[],[12],1
14,What is the currency of Brazil?,7,[13],[],[13],1
15,What gas do plants absorb during photosynthesis?,9,[14],[],[14],1
16,Who directed the movie Jaws?,7,[15],[],[15],1
17,What is the largest internal organ in the human body?,11,[16],[],[16],1
18,How many elements are in the periodic table?,9,[17],[],[17],1
19,What is the freezing point of mercury in Celsius?,10,[18],[],[18],1
20,Who painted The School of Athens?,7,[19],[],[19],1
21,What is the smallest unit of life?,8,[20],[],[20],1
22,How many time zones are there in Russia?,9,[21],[],[21],1
23,What is the main language spoken in Argentina?,9,[22],[],[22],1
24,Who invented the diesel engine?,6,[23],[],[23],1
25,What is the diameter of Earth in kilometers?,9,[24],[],[24],1
```

Simulation Pipeline

- **Purpose:** Enables RL training without physical hardware by mimicking system latency and memory constraints.
- **Core Architecture:** A dual-component Python system using a specialized Cache Simulator and a Gymnasium Wrapper.
- **Input Data:** Utilizes pre-generated Trace CSVs (50 sequential user queries per episode).
- **Key Outcome:** Evaluates "Time-To-First-Token" (TTFT) based on simulated millisecond bottlenecks.

Hardware Pipeline

- **The Concept:** A "drop-in" replacement for mathematical simulations that swaps theoretical latency for actual physical execution on hardware.
- **The 3-Tier Physical Memory:**
 - **L1 (GPU VRAM):** Real `torch.cuda.FloatTensor` objects for instant access.
 - **L2 (CPU Pinned RAM):** Tensors locked in physical memory using `pin_memory()` to enable high-speed Direct Memory Access (DMA).
 - **L3 (NVMe Disk):** Data stored as raw `.bin` files to mimic production C++ engine serialization.
- **The Goal:** Forces the AI to navigate real-world bottlenecks like PCIe bus saturation and SSD I/O limits that math alone cannot predict.

Testing

Standardized Proving Ground: Validates the trained agent against identical user traces under locked hardware constraints to quantify Time-To-First-Token (TTFT) savings.

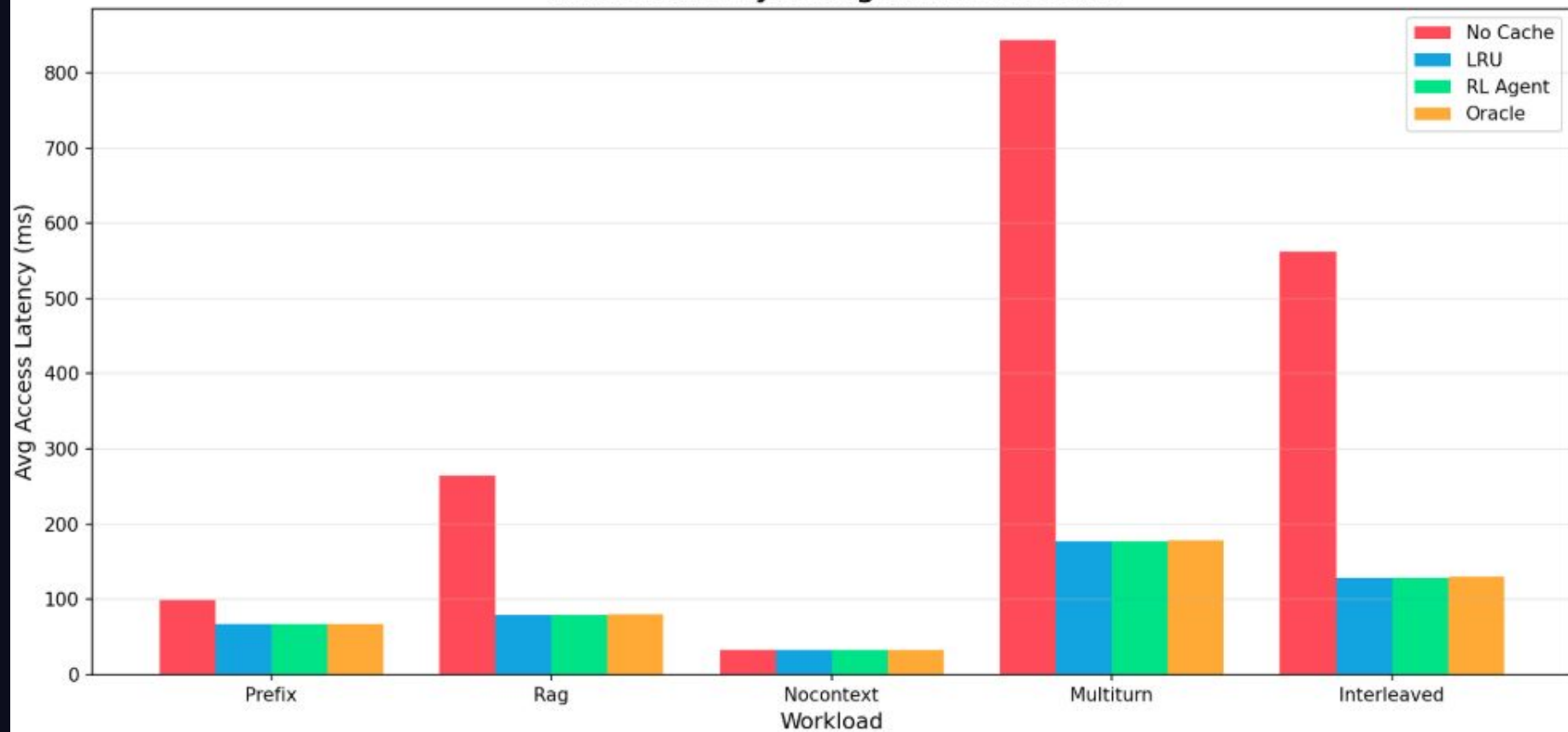
The Competition (Baselines):

- **No-Cache:** Baseline "worst-case" (recompute everything).
- **LRU (Passive):** Industry standard (passive eviction, no future-prediction).
- **Oracle (Ideal):** An "impossible" peak-performance limit used to measure how close the AI gets to "perfection."

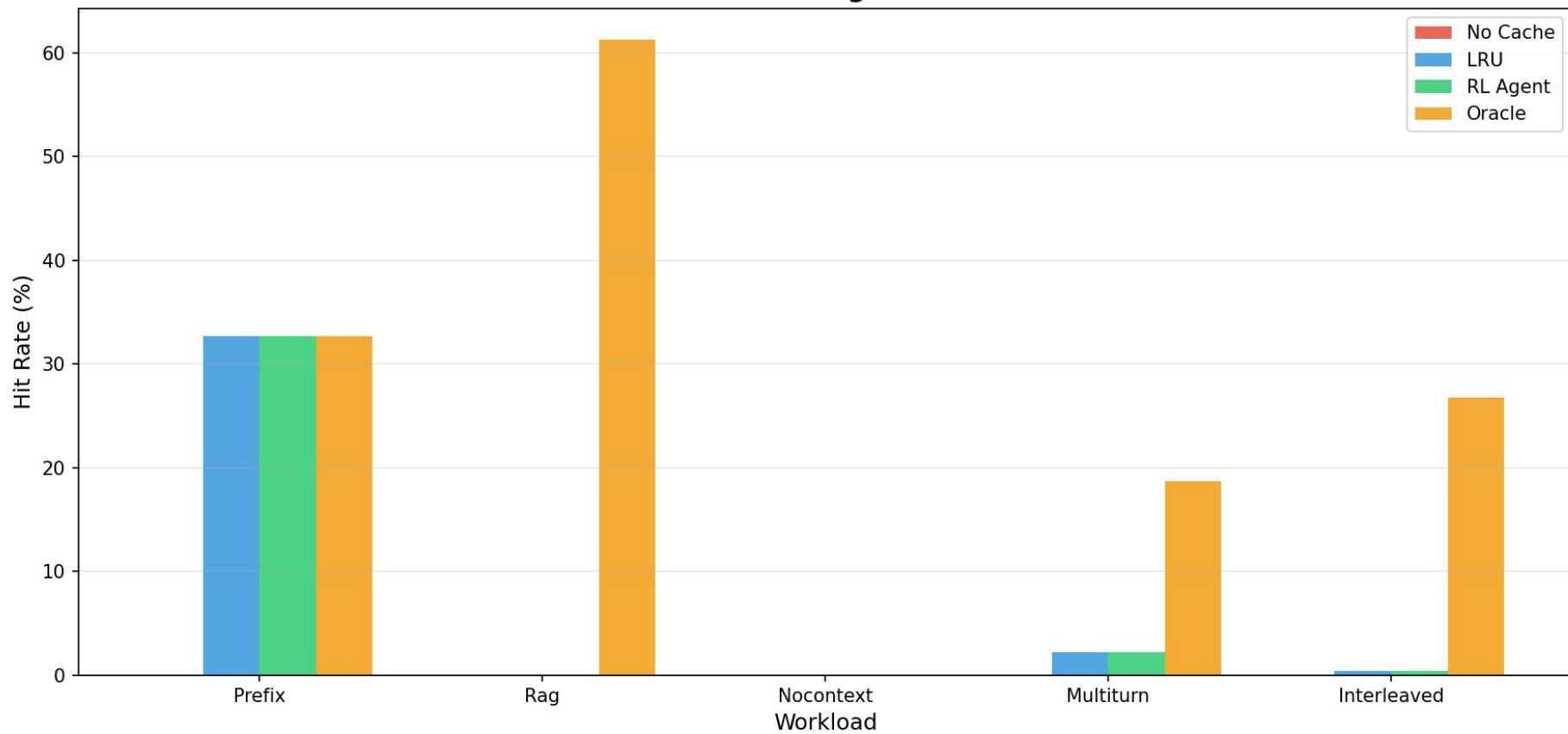
The tests are done over 5 tasks, Prefix, RAG, No context, Multiturn and interleaved.

Results

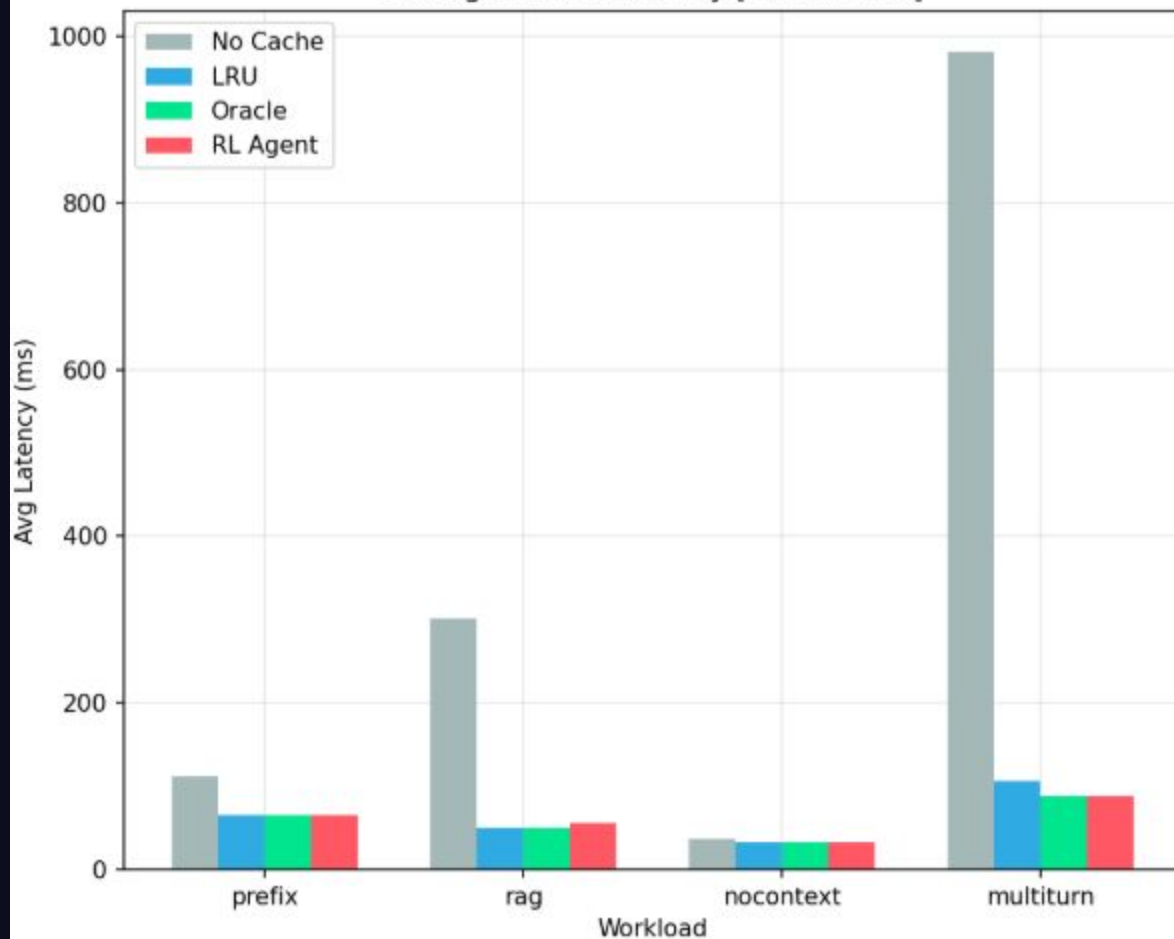
Access Latency: RL Agent vs Baselines

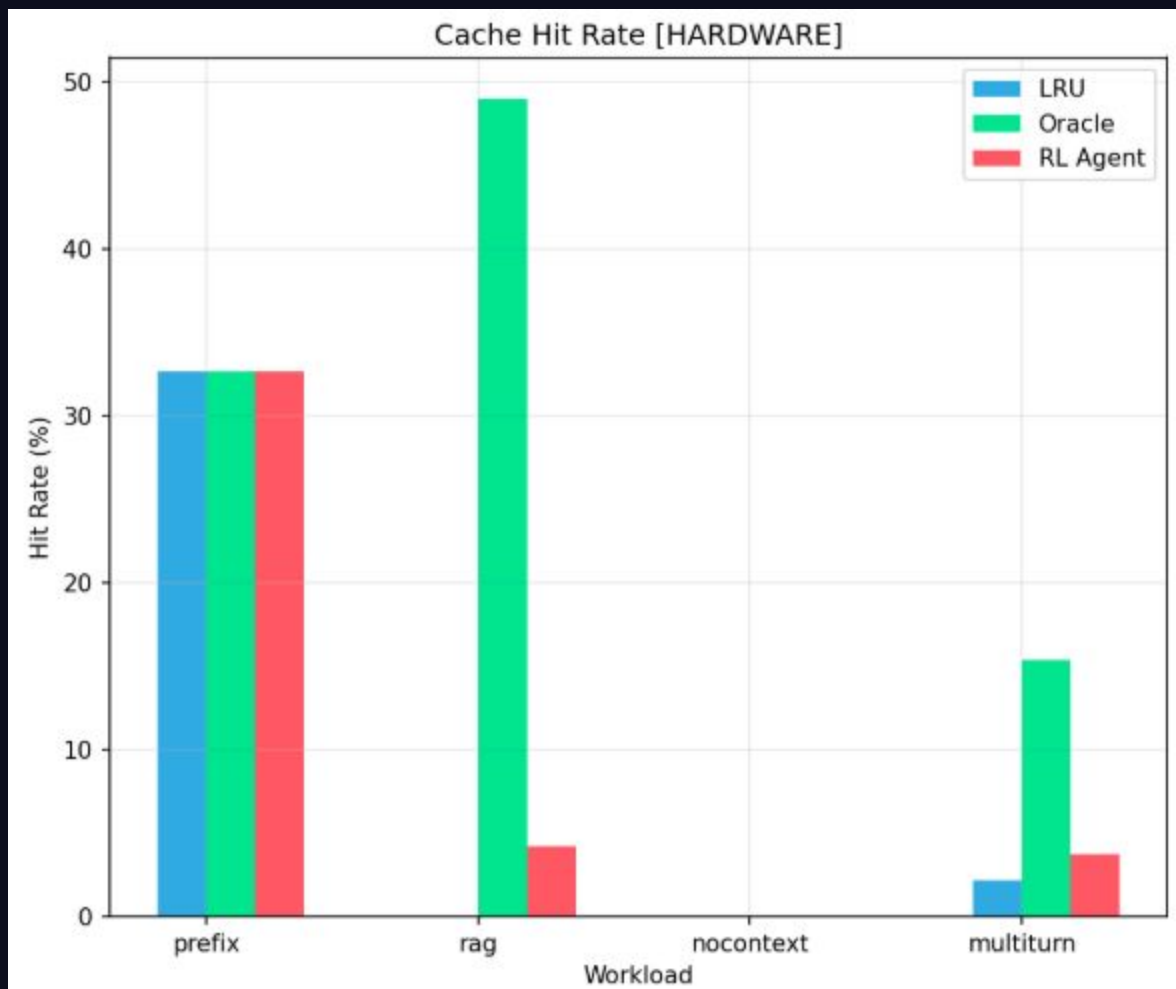


Cache Hit Rate: RL Agent vs Baselines

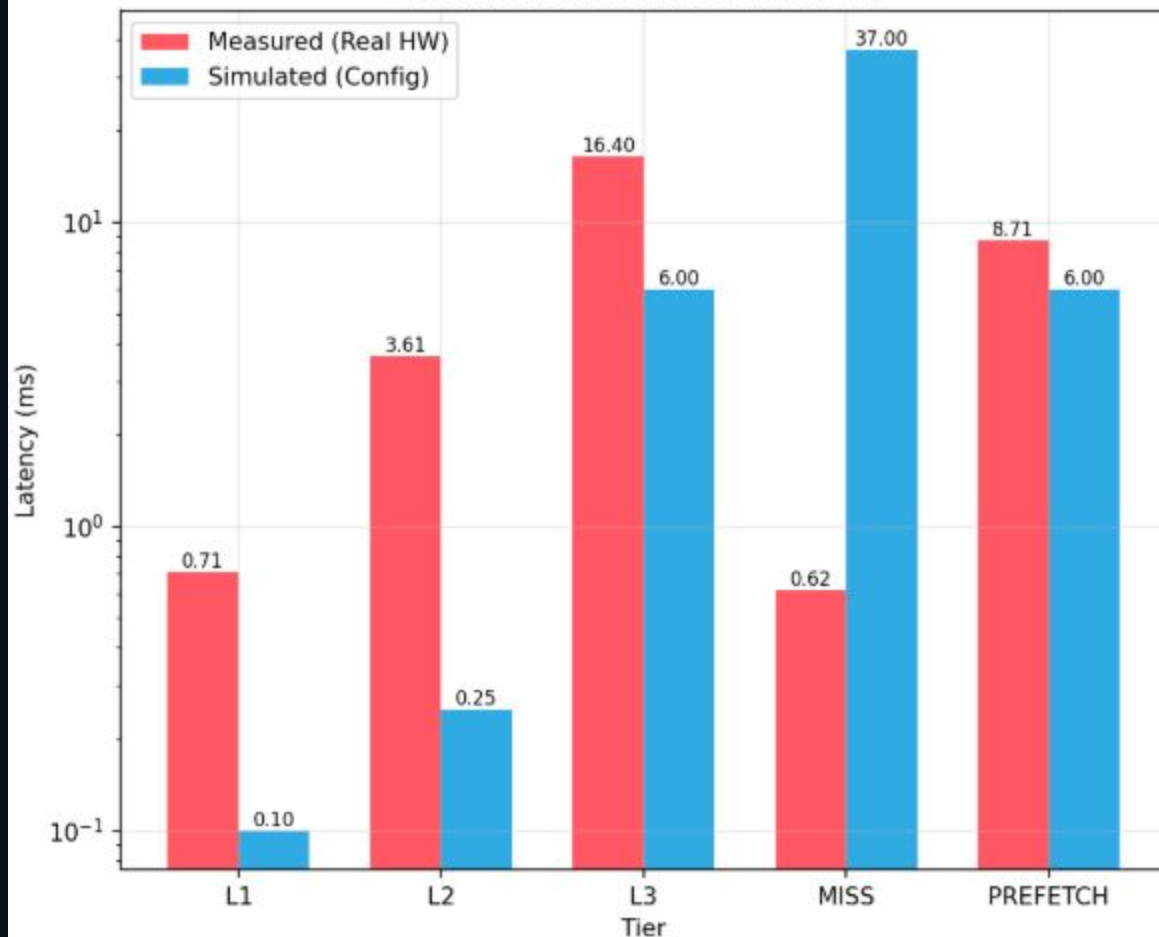


Average Access Latency [HARDWARE]





Measured vs Simulated Latencies



What we want to improve

- We want to make the `generate_traces` use actual GPUs
- Retest by changing the cost function
- Retest by changing the parameters of the cost function.
- Understand the results better. Why were the results they way they were.

Milestone 4: Verification & Decisions

- Take Decisions like how far to predict (risk vs accuracy), verifying migration waste (measuring `throughput_predictive` vs `throughput_baseline`). If the ratio < 1 : ML model is too heavy / inaccurate
- Performance Benchmarking:
 - Evaluate metrics like TTFT against Sequence Length (session size).
 - Calculate Hit Rate curve & comparing it with normal LRU cache or LM cache with “Neural-Prefetch” hit rate.
 - Calculate real-time inference time of the LSTM prefetcher and determine its effectiveness.
- Generating a final report of the findings and experiments.



Questions?

The background of the slide is a dark blue gradient with numerous thin, flowing, light blue lines that create a sense of movement and depth. These lines are concentrated in the lower half of the image, where they appear to originate from a point and fan out towards the edges. The overall effect is a dynamic and modern aesthetic.

Thankyou